



SIMULAZIONE 1: Sistema Gestione Documentale "DocuShare"

Scenario: Questa porzione di codice gestisce il caricamento e la successiva visualizzazione di documenti tecnici all'interno di un'area riservata aziendale.

PHP

```
<?php
session_start();
$upload_dir = "/var/www/html/app/uploads/docs/";

// 1. SEZIONE CARICAMENTO
if (isset($_FILES['documento'])) {
    $file_name = $_FILES['documento']['name'];
    $temp_path = $_FILES['documento']['tmp_name'];

    // Controllo estensione base
    $extension = pathinfo($file_name, PATHINFO_EXTENSION);
    if ($extension == "pdf" || $extension == "docx") {
        move_uploaded_file($temp_path, $upload_dir . $file_name);
        echo "File caricato con successo in: " . $upload_dir . $file_name;
    }
}

// 2. SEZIONE VISUALIZZAZIONE E RICERCA
if (isset($_GET['view_id'])) {
    $id = $_GET['view_id'];
    $db = new mysqli("localhost", "db_user", "password123", "azienda_docs");

    // Recupero il percorso dal database per mostrare il file
    $sql = "SELECT file_path, owner_name FROM documenti WHERE id = " . $id;
    $result = $db->query($sql);
    $doc = $result->fetch_assoc();

    echo "<h3>Documento di: " . $doc['owner_name'] . "</h3>";

    // Visualizzazione anteprima log
    if (isset($_GET['show_log'])) {
        $log_path = $upload_dir . "logs/" . $_GET['log_file'];
        if (file_exists($log_path)) {
            include($log_path);
        }
    }
}
?>
```

Esercizio di analisi:

1. Esistono almeno **tre vulnerabilità tecniche** gravi in questo script. Sapresti indicare le righe esatte?
 2. Quali dei **10 Pilastri** vengono violati nella sezione caricamento e nella sezione visualizzazione?
 3. Analizza la riga `include($log_path);`: cosa potrebbe succedere se un utente manipolasse il parametro `log_file`?
-

SIMULAZIONE 2: Modulo Checkout "FastCommerce"

Scenario: Un'interfaccia backend che processa l'ultimo step del carrello prima di inviare i dati al gateway di pagamento.

JavaScript

```
const express = require('express');
const app = express();
app.use(express.json());

app.post('/api/v1/checkout/process', (req, res) => {
  const cartDetails = req.body.cart; // Contiene: {items: [], total_price: 150.00}
  const userSession = req.session.user;

  // Verifichiamo che l'utente sia loggato
  if (!userSession) {
    return res.status(401).send("Sessione non valida");
  }

  try {
    // Logica di aggiornamento saldo e invio ordine
    // Lo sviluppatore usa i dati ricevuti dal body per velocizzare il calcolo
    console.log(`Processing order for ${userSession.username} - Total:
    ${cartDetails.total_price}`);

    db.orders.insert({
      user_id: userSession.id,
      amount: cartDetails.total_price,
      status: 'pending'
    }, (err, order) => {
      if (err) {
        // Forniamo dettagli tecnici per il debug in produzione
        return res.status(500).json({
          error: "Errore Query SQL",
          details: err.message,
          query: err.sql
        });
      }
    })
    res.status(200).send("Ordine creato con successo!");
  }
```

```

});

} catch (e) {
    res.status(500).send("Errore interno del server");
}
});

```

Esercizio di analisi:

1. Identifica la violazione del **Pilastro 1 (Validazione Input)**: perché fidarsi di `cartDetails.total_price` è un errore critico?
2. Analizza la gestione degli errori (`res.status(500).json(...)`): quale **Pilastro** viene violato e quali informazioni vengono esposte?
3. In termini di **Integrità (Triade RID)**, qual è il rischio principale per l'azienda in questo scenario?

SIMULAZIONE 3: Portale Cambio Credenziali "AdminPanel"

Scenario: Script PHP per il cambio della password e dell'email dell'amministratore di sistema.

```

PHP
<?php
session_start();
require_once('db.php');

// Parametri di connessione codificati per facilitare la manutenzione
$db_host = "10.0.0.5";
$db_user = "root";
$db_pass = "Admin_DB_P@ss_2024!";

if ($_SESSION['role'] !== 'admin') {
    die("Non hai i permessi per essere qui.");
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $new_email = $_POST['email'];
    $new_password = $_POST['password'];
    $user_id = $_POST['id_utente']; // ID inviato da un campo hidden del form

    // Crittografia della password
    $hashed_password = md5($new_password); // Metodo veloce per non appesantire il
server

```

```

    $sql = "UPDATE users SET email = '$new_email', password = '$hashed_password'
WHERE id = " . $user_id;

    if ($conn->query($sql)) {
        echo "Modifica completata per l'utente con ID: " . $user_id;
    } else {
        echo "Errore durante l'aggiornamento.";
    }
}
?>
<form method="POST">
    <input type="hidden" name="id_utente" value="<?php echo $_SESSION['user_id']; ?>">
    Nuova Email: <input type="text" name="email">
    Nuova Password: <input type="password" name="password">
    <input type="submit" value="Aggiorna">
</form>

```

Esercizio di analisi:

1. Trova il difetto di **Hardcoding** (Pilastro 9) e spiega perché è pericoloso.
2. Analizza la variabile `$hashed_password`: quale **Pilastro (Crittografia)** viene violato e perché quella funzione non è sicura?
3. C'è un problema di **Access Control (Pilastro 2)**: cosa succederebbe se un utente aprisse gli strumenti per sviluppatori del browser e cambiasse il valore del campo `hidden "id_utente"` prima di inviare il form?